

Lab experience 6: Digital I/O & Stepper motor

Objectives

- Gain experience with the implementation of digital logic in hardware
- Practice using digital output functions in LabVIEW or Arduino
- Use LabVIEW or Arduino to control a stepper motor

Reading & new concepts

Hardware

This lab exercise uses LabVIEW or Arduino to control a stepper motor via a motor driver circuit. This is an output-only activity; no data acquisition is required. If you are not familiar with stepper motors, it would be good to read one of the many online introductions, such as this one on the Adafruit web site:

[All About Stepper Motors | Adafruit Learning System](#)

The primary hardware for our digital control activity is a unipolar, 32-step motor with a step-down gearhead that brings the effective number of steps per shaft revolution to 513. It typically runs on a 5 V supply but can be supplied up to 9 V without damaging it. Although it is a “unipolar” motor, it actually has four control wires plus one common ground wire. The basic coil arrangement and technical description can be seen here:

[Small Reduction Stepper Motor - 5VDC 32-Step 1/16 Gearing](#)

To drive the motor, i.e. to provide the relatively high current that goes through each motor coil, we need one transistor per DAQ output channel. This current amplification is similar to the way you powered the LEDs in the pulse oximetry lab, except that motor control is typically done with MOSFETs rather than BJTs and the current requirements for a motor can be much higher than for an LED. We can either build a circuit with four MOSFETs or use an integrated driver circuit mounted on a printed circuit board (PCB). The PCB includes the integrated driver circuit (which contains four MOSFETs), plus the necessary resistors, capacitors, and connectors, as well as a voltage regulator that allows us to power the drive control circuit with 5 V, even though the integrated driver circuit requires 3.3 V. Each channel in the motor driver connects to one of the motor coils, so that when a particular output voltage from the Arduino or LabVIEW DAQ is high, current flows to that coil. Each digital channel will be represented by a Boolean variable in our VI or Arduino code and will be connected to four of the digital outputs. Motor drivers can be complicated to implement, but we plan to use drivers that are easy to use with our particular motors.

Software

Our stepper motor control software needs to produce the proper sequence of high and low voltages at the output terminals on an Arduino or USB-600x.

The controller is based on logical statements that determine the new state of each output based on the direction of rotation and the existing state of each LED. We will label the four outputs A, B, C, and D, such that the sequence is A-B-C-D-A-B-C... to rotate one way, and D-C-B-A-D-C-B... to rotate the other way. To make the motor operate more smoothly, we may include some “handoff” time when two succeeding coils are energized. In this case, the sequence goes

$A \rightarrow AB \rightarrow B \rightarrow BC \rightarrow C \rightarrow CD \rightarrow D$ etc. *or* $C \rightarrow BC \rightarrow B \rightarrow AB \rightarrow A \rightarrow AD \rightarrow D$ etc.

Prelab programming activity

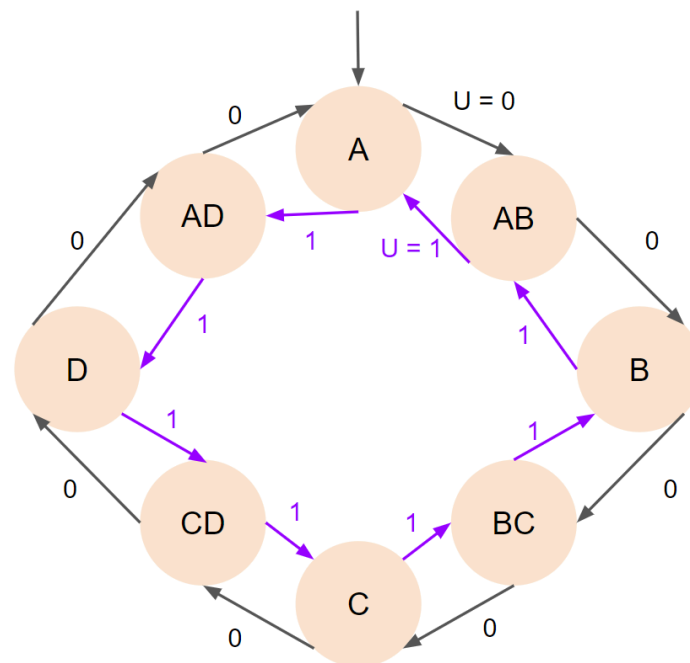


Figure 1. State-transition diagram of 4 LEDs with handoff period.

There are several ways to set the Boolean values for each cycle. Two practical options are (1) using the modulo (remainder) operator on the loop index `i`, and (2) using shift registers and logic statements to determine what the next value should be based on the current values. This handout assumes that we will use option (2).

1. Write logic statements for the four LED segments that simulate the stepper motor windings. These will have the form $\text{NextA} = f(\text{A}, \text{B}, \text{C}, \text{D}, \text{U})$, where U is the user input that determines direction. It is often helpful to sketch the state of each variable as a function of time to see how they transition. Feel free to use an online logic simplification tool like <http://www.32x8.com/index.html> (use the 5-variable option). The NextA column of the following state-transition table has been filled out

for you. Note that the table shows only 16 of the possible $2^5 = 32$ states, because we should never encounter them if we initialize our finite-state machine properly.

If you're confused, you may find **Appendix A Formula Node Equation Example** helpful.

Present State				User Input	Next State			
A	B	C	D	U	A	B	C	D
0	0	0	1	0	1			
0	0	0	1	1	0			
0	0	1	0	0	0			
0	0	1	0	1	0			
0	0	1	1	0	0			
0	0	1	1	1	0			
0	1	0	0	0	0			
0	1	0	0	1	1			
0	1	1	0	0	0			
0	1	1	0	1	0			
1	0	0	0	0	1			
1	0	0	0	1	1			
1	0	0	1	0	1			
1	0	0	1	1	0			
1	1	0	0	0	0			
1	1	0	0	1	1			

- NextA = $\sim C \sim B \sim U + \sim C \sim D U$
- NextB = _____
- NextC = _____
- NextD = _____

You will need to turn in the logic statements.

2. Using LabVIEW, create a VI that simulates the stepper motor coils with four LED (Boolean) indicators on the front panel. The VI should include these features:

- Control switches for direction and to pause operation (Run/Stop).
- A numeric control for motor speed. Speed can be controlled by using the numeric control to adjust the Wait or Wait Until Next Millisecond Multiple input. Note that since you are not sampling, consistent timing is not critical. **The shaft rotation speed should be limited to 25 rpm (not 25 Hz!).**
- Shift registers to pass the values of A, B, C, D from one loop to the next.
- A **formula node (recommended)** or set of Boolean logic operation icons, that implements the logic statements that you derived above. See this page for formula node syntax for binary operators: [Formula Node Syntax - NI](#)

You may wish to clean up your diagram (reduce the number of wires) by joining your A, B, C, & D wires into a 1-D array, which is represented by a single wire. An example is shown in Figure 3 at the end of Appendix 2.

Video of a finished front panel with working lights: <https://youtu.be/wkEyzIqJlvM>

Hardware design calculations

3. The Technical Details section of the stepper motor web page states that the DC resistance of each coil is $42\ \Omega$. **Calculate how much current each coil will draw** when 5 V is supplied to it, and how much total heat (power) the motor will dissipate during operation.

Bring your logic statements and VI to lab, where additional hardware details will be provided.

Arduino option: Simulate the coils with four physical LEDs. You may power the LEDs directly from the Arduino board as long as you have a sufficient resistance in series with each LED that you do not exceed 20 mA (this is a soft limit but you should consider the current limits of both the LED and Arduino before you select resistors). Speed can be controlled by means of a potentiometer connected between 5 V and ground, with the middle terminal connected to an analog input. A switch can control direction, but if you don't have one you can substitute a resistor connected from either 5 V (clockwise) or ground (counterclockwise) to one of the digital inputs. Note that you can determine the four state variables by making each one a number 1 or 0 and then using actual mathematical operations to perform the AND and OR operations; you can then input the 1 or 0 into the digitalWrite command to set the output to HIGH or LOW respectively (i.e., digitalWrite(<insert pin>, 1) or digitalWrite(<insert pin>, 0)).

Prototyping (In-lab activity)

The use of digital signals with our DAQ devices is described on pages 21-22 of the [USB-6001 user guide](#), with the terminal layout shown on page 9. Concepts and circuits presented in the Digital I/O section are explained in lecture.

4. Once you have shown that your VI can cycle the front panel LED on/off sequence properly, add a DAQ Assistant that takes in a 1-D array of logical values, and sends these four values to the digital channels P1.0, P1.1, P1.2 and P1.3. If you would prefer to use lower level functions (which is the more professional way to do it), please see the instructions provided in Appendix 2.

5. Plug the ULN2003 driver board into a power supply with limits of **5 V & 400 mA** and connect the P1.0, P1.1, P1.2, and P1.3 channels into the four pins on the board.

Note the order at which you are plugging each pin as it will correspond to the order of your array in DAQmx write. You will also need to connect the DAQ GND to the ULN2003 GND and Power supply GND. Run the VI.

6. Use a loop time of 50 ms. You should observe a slow but steady rotation in one direction when U is set to true, and rotation in the other direction when U is false.

7. Decrease the loop time until the motor stops rotating. Why do you think it stops?

Arduino option: At this time you may either switch to LabVIEW for step 8, or stay with Arduino by making a copy of your original A->AB->B->BC->C... code and revising it so it outputs to only two pins, following the A->AB->B->00->A->AB... pattern that is described in step 8 below for the downloadable VI.

8. Download the file **4 channels no handoff 2025.vi** from the Canvas assignment page. This VI works by alternating between the pink and orange pins while setting the yellow and blue pins to false. To rotate in the other direction, it alternates between the yellow and blue pins while setting the pink and orange pins to false. Connect the P1.0, P1.1, P1.2 and P1.3 to orange, pink, yellow, and blue pins, respectively. Run the VI.

Question: How is it possible that only two pins are used to rotate the motor. How is this possible?

9. Modify this downloaded VI to make the transition A->AB->B->00->A->AB... . There are a lot of good ways to do this, but one method is to use a Formula Node and a shift register (similar to what you did in prelab part 2) inside of a case structure. Run the test again. Is including handoff periods a good idea in this case? (Remember, because of the handoff period, the rotation rate is decreased by half.)

10. Demonstrate your VI and circuits to an instructor.

Canvas Submissions:

1. Upload your completed pre-lab VI or Arduino .ino file to Canvas. Each member of a team may submit the same VI.
2. There is no write-up for this lab, but you need to submit the logic statements you derived in Prelab Programming Question 1. Submit them using the text entry or as a comment.

Appendix 1: Formula Node Equation Example (3 channels)

Note that the logic formulas below are written in Boolean notation, which is not exactly the same as Formula Node syntax.

Example: Assume we have only three channels and no handoff period, and that $U=1$ (U is true) indicates clockwise motion. Then, to go clockwise, B should be set high in the following time step if A and U are high and C is low. We can write this as

$$B = A \& \sim C \& U, \text{ or just } A(\sim C)U$$

where $\sim$ indicates NOT and $\&$ indicates logical AND. If we are going counterclockwise, then B should be on if A and U are low and C is high, so the statement for B is

$$B = \sim A \& C \& \sim U.$$

This means B should turn on in either case, so we can write

$$B = A \& \sim C \& U \mid \sim A \& C \& \sim U, \text{ where } \mid \text{ indicates logical OR.}$$

We can also use the notation for multiply (AND) and add (OR):

$$B = A(\sim C)U + \sim A C \sim U = A(\sim C)U + \sim A C (\sim U).$$

In this way, you can use Arduino code or a Formula node to write your logical statements.

Appendix 2 Creating DAQmx output with lower level functions (not DAQ Assistant)

- Starting outside the while loop on the left-hand side, add DAQmx create virtual channel. Change **Analog Input** to **Digital Output**. Right click on the “lines” wire and create a control. Right click on the “line grouping” wire and create a constant. Set the constant to “one channel for each line”. This will allow us to have different outputs on different channels.
- Add a DAQmx Start function
 - Note from this point on, you will want to connect the task in/out and error/out sequentially (see image below)
- Add a DAQmx write function and change **Analog DBL 1Chan 1Samp** to **Digital 1D Bool NChan 1Samp**. Note that the input should be a 1D array since we are outputting to multiple channels, so make sure to use the “Build Array” function!
- Add another DAQmx write function **Digital 1D Bool NChan 1Samp**, and create a constant. Expand the array to show 4 points by dragging on the edge and set each point to false by clicking on the specific point.
- Add a DAQmx stop function followed by a DAQmx Clear function. At the end, right click on the error wire and create an indicator for troubleshooting.
- When you run the VI, make sure to specify P1 channels in the front panel.

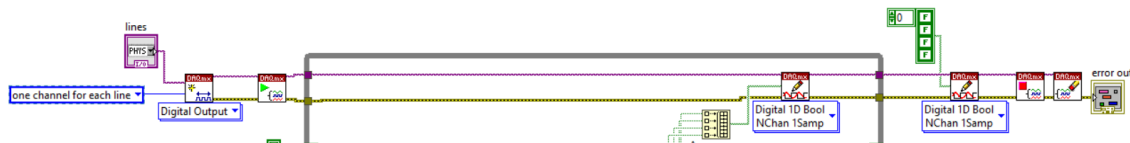


Figure 2. Diagram of DAQmx functions used to output data into digital channels.
Note: this is only a small portion of the VI.

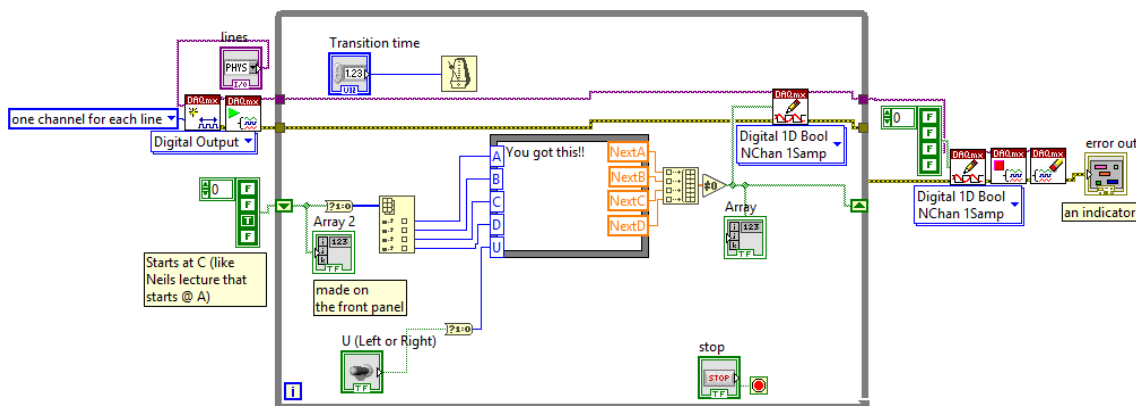


Figure 3. An example of a final VI using arrays to “clean up” the front panel.